Microsoft Fabric Learning Plan

Purpose

This document is the master learning and implementation plan for building a Microsoft Fabric platform from zero to production-ready maturity using an enterprise-minded, automation-first approach.

Delivery Principles

- CI/CD first
- CLI second
- Portal only as the last option
- all non-secret platform artifacts belong in source control
- prefer repeatable automation over one-time manual configuration

Project Scenario

Northwind Unified Commerce is the reference platform used across the program. The business needs a modern analytics platform for:

- batch sales, inventory, ERP, and supplier data
- near-real-time e-commerce and store telemetry
- governed semantic models and Power BI reporting
- platform administration, security, governance, and day-2 operations

Assumptions

- tenant region target is Canada Central unless a different region is required
- learning begins on a Fabric trial and later transitions to paid capacity
- Azure DevOps is the preferred Git and CI/CD platform for enterprise alignment
- Microsoft Entra ID groups are used for access management

Trial Constraints

- Fabric trial is time-limited and should be treated as a learning environment, not a production endpoint
- trial capacity does not represent the full enterprise networking posture
- some advanced capabilities are limited or unavailable in trial
- portal onboarding is still required for some initial activation tasks

Target Architecture

Logical Flow

1. Source systems land data into Fabric through pipelines, notebooks, and event ingestion.
2. Raw data lands in OneLake bronze.
3. Standardized transformations produce silver.
4. Curated gold data supports analytics and serving layers.
5. Lakehouse and Warehouse assets support data engineering and SQL-oriented access.
6. Semantic models expose governed business metrics.
7. Reports and dashboards serve business consumers.

8. Real-time analytics workloads cover telemetry and event use cases.

Core Platform Components

- OneLake
- Lakehouse
- Warehouse
- Pipelines
- Notebooks
- Eventstream
- KQL or Real-Time Intelligence assets where available
- Semantic models
- Reports and dashboards
- deployment pipelines
- Git integration

Operating Model

Environment Strategy

- dev for engineering and experimentation
- tst for integration and release validation
- prd for controlled production use

Workspace Strategy

- ‘ws-npw-platform-{env}‘
- ‘ws-npw-data-{env}‘
- ‘ws-npw-bi-{env}‘

In trial, the physical topology may be simplified, but the target design remains separated by ownership and lifecycle.

Administration Model

- Fabric admins own tenant settings and tenant-wide controls
- capacity admins own scale, monitoring, and workload behavior
- platform engineers own automation, workspace lifecycle, and guardrails
- domain or data product owners own business-aligned assets

Identity Model

- ‘sg-fabric-admins‘
- ‘sg-fabric-capacity-admins‘
- ‘sg-fabric-platform-engineers‘
- ‘sg-fabric-data-engineers‘
- ‘sg-fabric-bi-developers‘
- ‘sg-fabric-consumers‘
- ‘sg-fabric-breakglass‘
- ‘sg-fabric-automation-sp‘

Governance Model

- domains aligned to business capabilities, not tools
- naming standards and ownership metadata required
- endorsement, certification, and discoverability are designed early
- lineage and auditability are part of platform design, not afterthoughts

Security Model

- least privilege
- group-based access
- sensitivity labels where appropriate
- RLS and OLS on semantic models where required
- service principal access tightly scoped and allow-listed
- no secrets stored in source control

Networking Model

- learning mode uses public SaaS access where trial requires it
- production target uses Private Link for client-to-Fabric access where supported
- managed private endpoints and adjacent Azure services are used for source connectivity wher
supported
- gateway patterns are documented for hybrid access when relevant

Phased Roadmap

Phase 0: Program Foundation

Objective:
Define naming, branching, repo standards, architecture decisions, and a risk register before
workload creation begins.

Key outputs:

- repo structure
- delivery principles
- naming conventions
- initial backlog, risks, and technical debt logs

Primary methods:

- CI/CD
- CLI

Phase 1: Tenant, Admin, Identity, and Automation Baseline

Objective:
Stand up the Fabric learning environment foundation and establish repeatable platform
engineering practices.

Key outputs:

- trial onboarding

- tenant settings baseline
- Entra groups
- workspace strategy
- Git strategy
- first Fabric REST automation
- CI/CD starter pipeline

Primary methods:

- Portal for trial onboarding
- CLI for groups and API execution
- CI/CD for repo and pipeline foundations

Phase 2: Workspace and Release Foundation

Objective:
Provision workspaces, connect development workspaces to Git, and establish deployment pipelin
scaffolding.

Key outputs:

- dev workspaces
- Git-connected workspaces
- deployment pipeline structure
- environment parameter strategy

Primary methods:

- CLI
- CI/CD

Phase 3: OneLake and Core Data Platform Foundation

Objective:
Create the initial medallion-aligned data platform structure in Fabric.

Key outputs:

- Lakehouse foundation
- bronze, silver, gold layout
- first Warehouse or SQL-serving layer
- shortcut strategy where needed

Primary methods:

- CI/CD
- CLI

Phase 4: Batch Ingestion and Orchestration

Objective:
Bring source data into Fabric through controlled, repeatable ingestion patterns.

Key outputs:

- source landing patterns
- pipelines
- batch ingestion runbooks
- initial monitoring checks

Primary methods:

- CI/CD
- CLI

Phase 5: Transformation, Incremental Loads, and Data Quality

Objective:
Implement engineering logic, quality gates, and sustainable refresh patterns.

Key outputs:

- notebook or pipeline-based transformations
- incremental processing logic
- quality checks and failure handling
- medallion promotion logic

Primary methods:

- CI/CD
- CLI

Phase 6: Semantic Models, Security, and Reporting

Objective:
Deliver business-facing analytics with governed access and reusable metrics.

Key outputs:

- semantic models
- RLS and OLS where needed
- curated measures
- reports and dashboards

Primary methods:

- CI/CD
- Portal fallback where certain report authoring workflows are not fully automated

Phase 7: Real-Time Analytics

Objective:
Add streaming or near-real-time capabilities for telemetry and event-based use cases.

Key outputs:

- event ingestion pattern
- KQL or real-time analytics assets
- operational dashboards for telemetry

Primary methods:

- CI/CD
- CLI

Phase 8: Governance and Catalog Maturity

Objective:
Make the platform discoverable, auditable, and manageable at scale.

Key outputs:

- domains
- endorsement and certification model
- metadata standards
- ownership and stewardship mapping

Primary methods:

- CLI where supported
- Portal fallback for experiences without full automation coverage

Phase 9: Security Hardening and Network Maturity

Objective:
Move from learning-safe patterns to stronger enterprise controls.

Key outputs:

- target private connectivity design
- exfiltration risk reduction approach
- secrets and identity hardening
- logging and incident response hooks

Primary methods:

- CLI
- Portal
- adjacent Azure service integration

Phase 10: Capacity, Performance, and Operations

Objective:
Prepare the platform for sustained production operations.

Key outputs:

- capacity strategy
- performance monitoring
- alerting and runbooks
- support model and incident triage patterns

Primary methods:

- CLI
- Portal fallback

Phase 11: Production Readiness and Transition

Objective:
Validate that the platform can move from trial-era learning mode into a controlled enterprise delivery model.

Key outputs:

- production readiness checklist
- migration actions from trial to paid capacity
- remaining technical debt review
- platform operating model sign-off

Primary methods:

- CI/CD
- CLI
- controlled Portal actions where unavoidable

Current Backlog

- create remote Git repository
- make first commit and push baseline
- activate or verify Fabric trial
- create Entra groups
- capture Fabric tenant settings
- create the first development workspace
- validate service principal access for Fabric APIs
- connect workspace to Git
- create deployment pipeline shell

Risks and Tradeoffs

- trial capacity expires
- trial networking does not represent full production capability
- some APIs are preview and should be used carefully
- portal steps still exist in parts of Fabric administration
- simplified trial topology may diverge from target enterprise topology

Source Control Policy

Always store:

- manifests
- scripts
- pipeline YAML
- standards and decisions
- risk and technical debt logs
- runbooks and validation steps

Never store:

- access tokens
- client secrets
- certificate private keys
- plaintext credentials

What Success Looks Like

At the end of this journey, the platform should demonstrate:

- repeatable deployment foundations
- governed environments and workspaces
- secure, auditable access patterns
- end-to-end data ingestion and transformation
- governed analytics and reporting
- operational readiness for monitoring, support, and growth